

RELAZIONE DI PROGETTO

Sistema di Prenotazione Viaggi & Escursioni FSD

Applicazione Java (Swing) con architettura BCE e persistenza su PostgreSQL

Gruppo di lavoro

Poccia Fabio — matricola DE1000090

Ribera Davide — matricola DE1000220

Minopoli Simone — matricola DE1000388

1. Scopo del progetto

Il progetto ha l'obiettivo di realizzare un sistema software per la gestione della prenotazione di viaggi ed escursioni. Il sistema consente a un utente registrato di consultare un catalogo di attività turistiche organizzate per destinazione, effettuare prenotazioni, lasciare recensioni sulle attività svolte e comporre itinerari di viaggio personalizzati, articolati in tappe giornaliere.

Il dominio del problema è stato modellato attraverso un diagramma delle classi UML, che costituisce il punto di partenza sia per la progettazione del database relazionale sia per la struttura delle classi Java dell'applicazione. Il progetto nasce come esercizio di programmazione orientata agli oggetti e di basi di dati, con particolare attenzione alla coerenza tra il modello concettuale e l'implementazione.

1.1 Entità principali del dominio

Il modello concettuale individua le seguenti entità, ciascuna caratterizzata da attributi propri e da relazioni con le altre:

- Destinazione — luogo geografico che ospita una o più attività
- Attività — generalizzata in tre sottotipi: AttivitàEstrema, AttivitàCulturale, AttivitàNaturalistica
- GuidaLocale — conduce le attività ad essa assegnate
- Utente — gestisce il proprio profilo, effettua prenotazioni, scrive recensioni, crea itinerari
- Prenotazione — collega un utente a un'attività in una data specifica
- Recensione — valutazione numerica e testuale rilasciata da un utente su un'attività
- Itinerario — composto da una o più Tappe, entità debole esistente solo nel contesto dell'itinerario a cui appartiene

2. Architettura software: pattern BCE

L'applicazione segue il pattern architetturale Boundary-Control-Entity (BCE), organizzato nei seguenti package: gui (Boundary), controller (Control), model (Entity), dao e implementazionePostgresDAO (persistenza), database (gestione della connessione) ed Exception (eccezioni applicative).

2.1 Regola di dipendenza tra i livelli

Ogni package conosce e comunica esclusivamente con il livello immediatamente sottostante, senza saltare livelli intermedi:

- gui → controller: la GUI non chiama mai direttamente una DAO;
- controller → dao: il controller lavora contro le interfacce, mai contro le classi concrete;
- implementazionePostgresDAO → dao: le classi implementano le interfacce definite nel package dao;
- implementazionePostgresDAO → database: solo qui viene utilizzata la connessione JDBC per eseguire query;
- model: attraversa tutti i livelli, poiché rappresenta i dati;

2.2 Suddivisione delle responsabilità: DAO ed entità autonome

Il numero di interfacce DAO non coincide con il numero di classi del model, ma con il numero di entità dotate di vita autonoma nel database. Sono state realizzate 7 DAO: **Destinazione**, **GuidaLocale**, **Utente**, **Attività**, **Prenotazione**, **Recensione**, **Itinerario**. Le tre sottoclassi di Attività non hanno una DAO propria, in quanto rappresentano la stessa entità con dati aggiuntivi (gestite internamente da AttivitàDAO tramite il campo

discriminatore); analogamente Tappa, essendo un'entità debole priva di identità autonoma, viene gestita interamente all'interno di ItinerarioDAO.

2.3 Controller unico

Su indicazione del docente, il progetto adotta un controller unico anziché un controller per area funzionale. La classe Controller possiede un riferimento a ciascuna delle 7 DAO (istanziate una sola volta nel costruttore) ed espone, organizzati per sezioni commentate, i metodi che orchestrano le regole di business: calcolo del prezzo totale di una prenotazione, verifica della disponibilità dei posti, verifica delle credenziali di accesso, generazione dei codici identificativi, aggiornamento dei posti disponibili in seguito a una prenotazione o a un annullamento.

3. Flusso operativo: un esempio end-to-end

Per illustrare concretamente il funzionamento dell'architettura a livelli, si riporta il percorso completo di un'operazione tipica: la creazione di una nuova prenotazione a partire dal catalogo delle attività.

```

GUI (Catalogo)
  utente seleziona un'attività e un numero di posti
  click su "Conferma prenotazione"
    |
    v
Controller.creaPrenotazione(idUtente, codiceAttivita, posti, data)
  1. recupera l'attività con AttivitaDAO.trovaPerCodice(...)
  2. verifica la disponibilita' dei posti (regola di business)
  3. calcola il prezzo totale = prezzo unitario * numero posti
  4. costruisce l'oggetto Prenotazione (model)
    |
    v
PrenotazioneDAO.inserisci(prenotazione)
  |
  v
PrenotazioneDAOImpl
  costruisce la query INSERT con PreparedStatement
  esegue la query tramite ConnessioneDatabase (singleton)
    |
    v
PostgreSQL – riga inserita nella tabella prenotazione
  |
  v
Controller aggiorna i posti disponibili dell'attività
  tramite AttivitaDAO.aggiornaPostiDisponibili(...)
    |
    v
GUI (AreaPersonale) – al successivo accesso alla sezione
  "Le mie prenotazioni", i dati vengono ricaricati dal database
  e la nuova prenotazione compare automaticamente
  
```

Questo esempio evidenzia due principi seguiti sistematicamente in tutto il progetto: la logica di business (calcolo del prezzo, verifica dei posti, generazione dei codici) risiede esclusivamente nel controller, mai nella DAO né nella GUI; e ogni finestra dell'interfaccia grafica ricarica i propri dati dal database ogni volta che l'utente torna a consultarla, così da restare sempre sincronizzata con lo stato reale del sistema senza necessità di comunicazione diretta tra le diverse finestre.

4. Funzionalità implementate

4.1 Autenticazione e profilo

- Registrazione di un nuovo utente, con controllo di unicità sull'indirizzo email
- Accesso tramite email e password
- Visualizzazione e modifica dei propri dati anagrafici e di accesso.

4.2 Catalogo attività

- Consultazione delle attività disponibili, filtrabili per destinazione, categoria e fascia di prezzo
- Visualizzazione del dettaglio di un'attività: descrizione, prezzo, guida locale associata.

4.3 Prenotazioni

- Creazione di una prenotazione con calcolo automatico del prezzo totale
- Aggiornamento automatico dei posti disponibili dell'attività
- Consultazione delle proprie prenotazioni e annullamento di una prenotazione esistente

4.4 Recensioni

- Scrittura di una recensione (testo e voto da 1 a 5) su un'attività prenotata
- Consultazione delle proprie recensioni

4.5 Itinerari

- Creazione di un itinerario personalizzato
- Gestione delle tappe: aggiunta e rimozione
- Consultazione dei propri itinerari con il relativo dettaglio delle tappe

5. Conclusioni

Il progetto ha permesso di applicare in un caso di studio realistico l'intero percorso che porta da un modello concettuale UML a un sistema software funzionante, curando la coerenza tra le tre rappresentazioni del dominio: il diagramma delle classi, lo schema relazionale del database e le classi Java dell'applicazione. Le scelte progettuali più significative — la traduzione della generalizzazione tramite ereditarietà per tabelle, l'uso del codice di dominio come chiave primaria dove previsto dall'UML, la gestione delle entità deboli e la rigida separazione delle responsabilità tra i package secondo il pattern BCE — sono state motivate passo per passo, con l'obiettivo di produrre un'architettura non solo funzionante, ma anche comprensibile e giustificabile nelle sue scelte.